PRACTICE II

Literature retrieval

1. Introduction

- Public-domain digital libraries make it possible to build literary collections for computational analysis.
- Literary works can be segmented into chapters or fragments to create searchable document units.
- A retrieval system makes it possible to locate passages related to a theme, situation, character, setting, or literary motif.

2. Objective

Develop a program that collects public-domain literary works, segments them into document units, and allows users to search for and retrieve the most relevant passages for a text query using VSM and cosine similarity.

3. General specifications

- Work in teams of 3-4 members.
- Collect and structure your own corpus from the indicated or approved sources.
- Apply text normalization using spaCy, NLTK, or another justified library.
- Represent the documents using the Vector Space Model with scikit-learn or an equivalent implementation.
- Generate frequency, binary, and TF-IDF representations.
- Experiment with unigrams and bigrams.
- Retrieve the documents most similar to a query using cosine similarity.
- Build a benchmark of queries and relevant documents and quantitatively evaluate the system.

4. Data collection

Collect public-domain literary works from Project Gutenberg or another instructor-approved collection. Use multiple authors and works, and define a reproducible segmentation strategy so that chapters or fragments can be treated as retrieval documents.

Item	Specification
Volume	Recommended: 300-500 documents
Sources	Project Gutenberg and/or approved public-domain collections
Document unit	Chapter or fragment; justify the segmentation choice

Diversity	Report distribution by author, work, and language/category when applicable

Good data acquisition practices:

- Respect access conditions, request limits, and the source's public access mechanisms.
- Do not bypass CAPTCHA, authentication, or anti-scraping mechanisms.
- Log acquisition errors and preserve traceability to the original source.
- When feasible, preserve both a raw and a structured version of the corpus.

5. Raw corpus

The corpus must contain at least the following fields:

Field	Description
fragment_id	Unique fragment identifier
book_id	Work identifier
title	Work title
author	Author
chapter/position	Chapter or position in the work
text	Fragment text
language	Language
source	Source collection
url	Original link if available

The raw corpus must be saved in a TSV (Tab-Separated Values) file called literature_raw_corpus.tsv.

6. Text normalization

Apply the normalization process to: Title and Fragment Text.

- Tokenization.
- Remove stopwords from the following grammatical categories: articles, prepositions, conjunctions and, pronouns. POS tagging process should be used to identify grammatical categories.
- Lemmatization.
- Normalize whitespace and control characters.

The normalized version of the corpus must be saved as literature_normalized_corpus.tsv.

7. Vector Space Model representation

Generate vector representations for: Title and Fragment Text.

Representation	Features
Frequency	Unigrams and bigrams
Binary	Unigrams and bigrams
TF-IDF	Unigrams and bigrams

The matrices and vectorizers may be stored in PKL files.

8. Information retrieval

The query will be free text describing a theme, situation, character, setting, or literary motif.
Examples: “grief and loss”, “an argument between a father and son”, “description of a storm”.

For each query, the following must be indicated:

- The comparative content of the literary document (Title or Fragment Text)
- The features to be extracted (unigram or Bigram)
- The type of vector representation (frequency, binary or TF-IDF)

The following process must be performed for each query:

- Apply exactly the same normalization process used to the normalized corpus
- Extract the specified feature
- Represent the query in the specified vector space
- Calculate cosine similarity between the query and the documents using the specified content
- Sort the results in descending order
- Display the 10 most similar documents, including the score

9. Benchmark and evaluation

Build a set of **15** representative queries. The queries have **3** levels of complexity:

- Easy. Strong lexical overlap
- Medium. Some lexical variation
- Hard. Semantic relationship with limited lexical overlap

Create **5** easy, **5** medium and **5** hard queries.

Examples of levels of complexity

Level	query	Relevant documents
Easy	description of a storm	Relevant passages will probably contain words such as storm, rain, wind, thunder, or lightning.
Medium	grief after the death of a loved one	Relevant passages might describe mourning, sorrow, loss, or bereavement without using the exact query wording.
Hard	a character realizes that ambition has isolated them from others	A relevant passage may express loneliness, regret, or the consequences of ambition without explicitly naming those concepts.

For each query, a set of **10–15** relevant documents must be identified in advance. Relevance is rated on a **3-point** scale. Sets must contain a combination of relevant documents with values of 1 and 2; those with a value of 0 are excluded.

Value	Meaning
0	Not relevant
1	Partially relevant
2	Highly relevant

The benchmark must be frozen and saved in two TSV files.

- queries.tsv
- qrels.tsv

The formats of the tables is explained below.

queries.tsv format

query_id	difficulty_level	query
Q01	1	description of a storm
Q02	1	a conversation about marriage
Q03	2	grief after the death of a loved one
... Q15	 3	... a character realizes that ambition has isolated them from others

qrels.tsv format

query_id	document_id	relevance
Q01	3	2
Q01	18	2
Q01	2	1
... Q15	 45	... 1

The ranking predictions for each query must be saved in a TSV file named rankings.tsv. The file has the following structure.

query_id	document_id	rank	score
Q01	3	1	0.83
Q01	18	2	0.81
Q01	2	3	0.79
Q01	45	4	0.76
Q01	4	5	0.73
Q01	12	6	0.69
Q01	18	7	0.66
Q01	7	8	0.53
Q01	34	9	0.45
Q01	123	10	0.42
Q02	201	1	0.90
...	...	...	...
Q15	89	10	0.41

The results of the document retrieval process will be evaluated using the following metrics.

Metric	Description	Interpretation
Precision@10 (P@10)	Proportion of relevant documents among the first 10 retrieved documents. For this metric, documents with relevance `1` or `2` are considered relevant.	A value of `0.80` means that 8 out of the 10 retrieved documents are relevant. A higher value indicates greater precision in the results presented to the user.
Recall@10 (R@10)	Proportion of all relevant documents identified in the benchmark that appear among the first 10 retrieved documents. Documents with relevance `1` or `2` are considered relevant.	If there are 12 relevant documents for a query and 9 appear in the Top-10, then `R@10 = 0.75`. A higher value indicates better coverage of the relevant documents.
F1@10	Harmonic mean of Precision@10 and Recall@10. It provides a single measure that balances the precision and coverage of the retrieval system.	A high value requires both Precision@10 and Recall@10 to be high. The maximum possible value is `1.0`.
MRR (Mean Reciprocal Rank)	Measures how early the first relevant document appears in the ranking. For each query, the reciprocal of the rank of the first relevant document is calculated (`RR = 1/rank`). MRR is the average RR across the 15 benchmark queries.	If the first relevant document appears at ranks `1`, `2`, or `5`, the corresponding RR values are `1.0`, `0.5`, and `0.2`. A higher MRR indicates that the system tends to place at least one relevant document near the top of the ranking.
nDCG@10 (Normalized Discounted Cumulative Gain)	Evaluates the quality of the Top-10 ranking by considering both the position and the degree of relevance (`0` = not relevant, `1` = partially relevant, `2` = highly relevant). Relevant documents appearing at higher ranks receive greater importance.	A value close to `1.0` indicates that the ranking produced by the system is close to the ideal ranking, with highly relevant documents appearing before partially relevant or non-relevant documents.

Example: Evaluation of Information Retrieval Metrics

This example illustrates how to compute the evaluation metrics for a single query (Q01) using the files **qrels.tsv** and **rankings.tsv**. The same procedure is applied independently to each of the 15 queries in the benchmark.

1. Input Files for the Example

1.1 qrels.tsv (Relevance judgments)

The file **qrels.tsv** contains the documents judged as relevant for query Q01.

query_id	document_id	relevance
Q01	N003	2
Q01	N007	1
Q01	N012	2
Q01	N018	1
Q01	N024	2
Q01	N031	1

- Relevance values:
  0 = not relevant
  1 = partially relevant
  2 = highly relevant
- For Q01 there are 6 relevant documents (relevance > 0).

1.2 rankings.tsv (System results)

The file **rankings.tsv** contains the top-10 documents retrieved by the system for query Q01.

query_id	document_id	rank	score
Q01	N003	1	0.910
Q01	N050	2	0.850
Q01	N012	3	0.820
Q01	N018	4	0.790
Q01	N060	5	0.720
Q01	N024	6	0.690
Q01	N070	7	0.610
Q01	N031	8	0.580
Q01	N080	9	0.510
Q01	N090	10	0.470

2. Combined View for Q01

The following table shows the top-10 retrieved documents for Q01 with their corresponding relevance labels (documents not listed in **qrels.tsv** are considered to have relevance 0).

Rank	Document	Score	Relevance	Relevant?
1	N003	0.910	2	Yes
2	N050	0.850	0	No
3	N012	0.820	2	Yes
4	N018	0.790	1	Yes
5	N060	0.720	0	No
6	N024	0.690	2	Yes
7	N070	0.610	0	No
8	N031	0.580	1	Yes
9	N080	0.510	0	No
10	N090	0.470	0	No

3. Evaluation Metrics

Below are the equations and step-by-step calculations for each metric using the example of query Q01.

3.1 Precision@10 (P@10)

Proportion of relevant documents among the top-10 retrieved documents.

$$P@10 = \frac{1}{10}\sum_{i=1}^{10} rel(d_i) \qquad \text{where } rel(d_i) = \begin{cases} 1, & \text{if relevance}(d_i) > 0 \\ 0, & \text{otherwise} \end{cases}$$

For Q01, 5 of the 10 retrieved documents are relevant:

$$P@10 = \frac{5}{10} = 0.50$$

3.2 Recall@10 (R@10)

Proportion of all relevant documents that are retrieved in the top-10.

$$R@10 = \frac{\sum_{i=1}^{10} rel(d_i)}{|\{d \in Q_{rel}\}|}$$

where Q_{rel} is the set of all relevant documents for the query (relevance > 0).

For Q01, there are 6 relevant documents in total, and 5 are retrieved:

$$R@10 = \frac{5}{6} = 0.833$$

3.3 F1@10

Harmonic mean of Precision@10 and Recall@10.

$$F1@10 = 2 \cdot \frac{P@10 \cdot R@10}{P@10 + R@10}$$

For Q01:

$$F1@10 = 2 \cdot \frac{0.50 \cdot 0.833}{0.50 + 0.833} = 0.625$$

3.4 Reciprocal Rank (RR)

Inverse of the rank of the first relevant document.

$$RR = \frac{1}{\text{rank}_1}$$

where $rank_1$ is the position of the first relevant document (relevance > 0).

For Q01, the first relevant document (N003) appears at rank 1:

$$RR = \frac{1}{1} = 1.0$$

The Mean Reciprocal Rank (*MRR*) is the average of the *RR* values over the 15 queries:

$$MRR = \frac{1}{15}\sum_{q=1}^{15} RR_q$$

3.5 Normalized Discounted Cumulative Gain (nDCG@10)

Evaluates the quality of the ranking considering the position and the graded relevance values.

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{rel(d_i)} - 1}{\log_2(i+1)}$$

$$nDCG@10 = \frac{DCG@10}{IDCG@10}$$

Rank (i)	1	2	3	4	5	6	7	8	9	10
Relevance ($rel(d_i)$)	2	0	2	1	0	2	0	1	0	0
Gain ($2^{rel}-1$)	3	0	3	1	0	3	0	1	0	0
Discount $\log_2(i+1)$	1.000	1.585	2.000	2.322	2.585	2.807	3.000	3.170	3.322	3.459
Contribution	3.000	0.000	1.500	0.431	0.000	1.068	0.000	0.315	0.000	0.000

$DCG@10 = 3.000 + 0 + 1.500 + 0.431 + 0 + 1.068 + 0 + 0.315 + 0 + 0 =$ **6.314**

The ideal ranking places the 6 relevant documents in decreasing order of relevance: [2, 2, 2, 1, 1, 1, 0, 0, 0, 0].

Rank (i)	1	2	3	4	5	6	7	8	9	10
Ideal relevance)	2	2	2	1	1	1	0	0	0	0
Gain ($2^{rel}-1$)	3	3	3	1	1	1	0	0	0	0
Discount $\log_2(i+1)$	1.000	1.585	2.000	2.322	2.585	2.807	3.000	3.170	3.322	3.459
Contribution	3.000	1.893	1.500	0.431	0.387	0.556	0.000	0.000	0.000	0.000

$IDCG@10 = 3.000 + 1.893 + 1.500 + 0.431 + 0.387 + 0.356 + 0 + 0 + 0 + 0 =$ **7.567**

$nDCG@10 = 6.314 / 7.567 = 0.835$

10. Interface

The application must provide clearly separated interfaces or commands for the following tasks:

- Corpus collection/update.
- Normalization and representation generation.
- Run a query and retrieve the Top-10 results.
- Run the benchmark and report the metrics.

11. Evidence

- Source code.
- Raw corpus.
- Normalized corpus.
- Vector representation files/models.
- Benchmark file: queries and relevant documents.
- Evaluation results per query and averages.
- PDF document with results, error analysis, and team member names.

12. Error analysis

Select at least five queries with good performance and five with poor performance. Explain the observed causes: lack of lexical overlap, synonyms, overly frequent terms, ambiguous queries, inadequate normalization, proper names, domain-specific vocabulary, or other factors. These cases will be retained for later comparison with embedding-based retrieval.

13. Completion criterion

The practice is complete when the team can update and segment the corpus reproducibly, generate its representations, enter a free-text literary query, display a Top-10 ranking of passages, and report metrics on the benchmark.